

- 1> (b) Data and a pointer to the next node.
- 2> (c) NULL
- 3> (b) Self-referential structure
- 4> (b) head == NULL
- 5> (b) newnode -> ~~head~~ next = head; head = newnode;
- 6> (b) temp = head; head = head -> next; free(temp);
- 7> (c) Traversing from ~~fast~~ tail to head.
- 8> (b) n
- 9> (c) Copying the data of p -> next into P, then deleting p -> next.
- 10> (b) while (p != NULL)
- 11> (b) ~~p == head~~ p -> next == NULL
- 12> (c) Two pointers advancing at different rates
- 13> (c) Three
- 14> (b) Exactly one node.
- 15> (b) P = (struct node*) malloc(sizeof(struct node));
- 16> (b) Avoid special-case handling for insertion and deletion at the head.
- 17> (c) The first node.
- 18> (b) It does not support random
- 19> (b) Loss of access to all nodes, causing a memory leak.
- 20> (b) A slow pointer and a fast pointer.
- 21> (c) The last node.

22) (b) Nodes may be scattered anywhere in memory.

```
23) struct node {
    int data;
    struct node * next;
};
```

24) The code prints the sum of each node and its next node:

$$\rightarrow 10 + 20 = 30$$

$$\rightarrow 20 + 30 = 50$$

$$\rightarrow 30 + 40 = 70$$

• output is $\rightarrow 30 \quad 50 \quad 70$

25) Error:- head is passing by value, so changes to the head pointer won't affect the original list. Also, an empty-list case needs handling.

□ correct version:-

```
void insertEnd (struct node ** head, int x)
{
    struct node * t = (struct node *) malloc
        (Size of (struct node));
```

```
    t->data = x;
```

```
    t->next = NULL;
```



```
if (*head == NULL) {
```

```
    *head = t;
```

```
    return;
```

```
}
```

```
struct node *p = *head;
```

```
while (p->next != NULL)
```

```
    p = p->next;
```

```
    p->next = t;
```

```
}
```

26) struct node * reverse (struct node * head) {

```
    struct node * prev = NULL;
```

```
    struct node * curr = head;
```

```
    struct node * next = NULL;
```

```
    while (curr != NULL) {
```

```
        next = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
}
```

```
    return prev;
```

```
}
```

27) Suppose, the list is $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \Rightarrow \text{NULL}$

2) I want to delete 30.

$\Rightarrow$ Then, first 20 node's next pointer needs to be pointed to 40.

Before $\rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$

Prev del next

After $\rightarrow 10 \rightarrow 20 \rightarrow \dots \rightarrow 40 \rightarrow \dots \rightarrow \text{NULL}$

$\uparrow$

30 deleted.

$\# \text{ prev} \rightarrow \text{next} = \text{del} \rightarrow \text{next};$
 $\text{free}(\text{del});$

28) Initial list :-

$5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(i) $7 \rightarrow 5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(ii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow \text{NULL}$

(iii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{NULL}$

Final answer :- $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{NULL}$